

課題⑤ iptables によるポート転送

目的

iptables を用いたパケットの制御の実習を行う。パケットを分析することによりポート転送の方法を確認する。

事前準備

1. server と router の VM を起動する。
2. iptables コマンドについて調査する。

手順

1. router で dropbear が起動していないことを確認し、起動している場合は停止する。

図 1 は router で dropbear が起動していないことを確認する様子である。

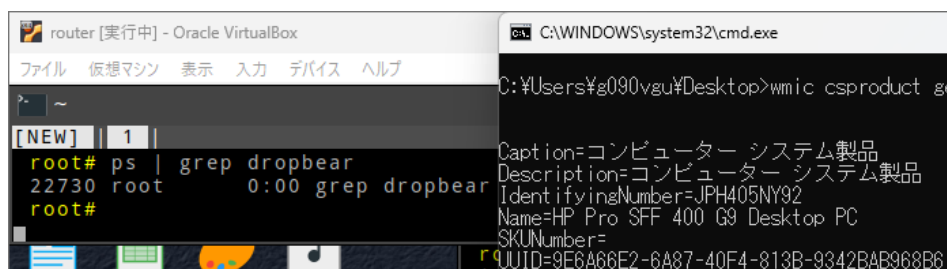


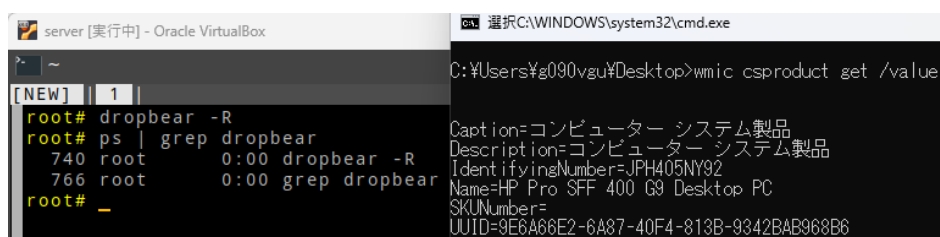
図 1 server で dropbear が起動していないことの確認

ps コマンドで プロセス一覧を取得し、その結果を grep コマンドで dropbear という文字列を含むものに限定している。フィルタリングされたプロセス一覧には、grep dropbear(プロセス一覧からの文字列検索に用いたコマンド) 以外のコマンドによるプロセスは表示されていない。したがって、router において dropbear が起

動していないことが確認できる。

2. server で dropbear を -R オプション付きで起動し，起動を確認する。

図 2 は server で dropbear を -R オプション付きで起動し，起動を確認する様子である。



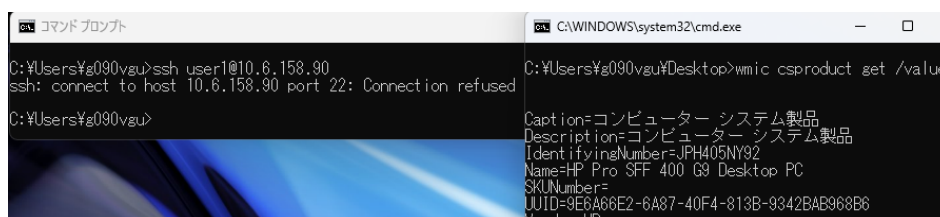
```
server [実行中] - Oracle VirtualBox
[NEW] 1
root# dropbear -R
root# ps | grep dropbear
740 root      0:00 dropbear -R
766 root      0:00 grep dropbear
root# _
```

図 2 server で dropbear 起動確認

ここではプロセス一覧において dropbear -R というコマンドで起動されたプロセスが表示されていることから，dropbear が-R オプション付きで起動していることが確認できる。

3. デスクトップ PC(Windows) から router へ ssh で一般権限ユーザでログインできないことを確認する。

図 3 はデスクトップ PC から router への ssh 接続に失敗する様子である。



```
コマンドプロンプト
C:\Users\%g090vgu>ssh user1@10.6.158.90
ssh: connect to host 10.6.158.90 port 22: Connection refused
C:\Users\%g090vgu>
```

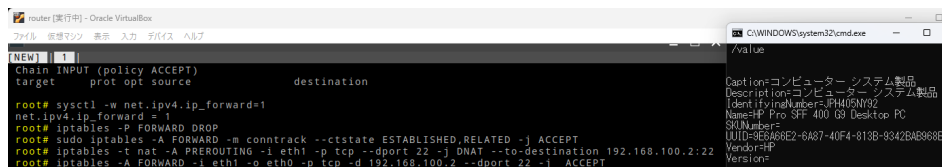
図 3 デスクトップ PC から router への ssh 接続に失敗する様子

ssh: connect to host 10.6.158.90 port 22: Connection refused というエラーメッセージが表示されている。手順 1 で確認したとおり，router においては dropbear のプロセスが起動されていないためである。

4. router の組織外側インターフェースに対して以下の iptables 設定を行う（必要なら sysctl で転送を有効化：net.ipv4.ip_forward=1）：

- ① TCP 22 番ポート宛てのパケットを server の TCP 22 番ポートへ転送する（例：PREROUTING で DNAT を設定）。
- ② TCP のそれ以外のポート宛てアクセスは転送しない（DROP/REJECT を設定）。

図 4 は手順 4 の要件を満たすような iptables の設定を行うコマンドの実行結果である。



```
root@router:~# sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
root@router:~# iptables -P FORWARD DROP
root@router:~# sudo iptables -A FORWARD -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
root@router:~# iptables -t nat -A PREROUTING -i eth1 -p tcp --dport 22 -j DNAT --to-destination 192.168.100.2:22
root@router:~# iptables -A FORWARD -i eth1 -o eth0 -p tcp -d 192.168.100.2 --dport 22 -j ACCEPT
```

図 4 手順 4 の要件を満たすための iptables の編集コマンド

ここで行った設定についてコマンドごとに説明を行う。

```
# sysctl -w net.ipv4.ip_forward=1
```

sysctl コマンド [1] を -w オプションで用いて、net.ipv4.ip_forward の値を 1 に設定することで、ポートフォワーディングを有効化した。

```
# iptables -P FORWARD DROP
```

明示的に許可した通信以外は転送しないようにするため、-P オプションによって対象のチェーンのポリシーを設定している。ここでは filter テーブル上の FORWARD チェインに対して DROP というポリシーを設定し、デフォルトですべてのパケットを転送できないよう設定している [3]。

```
# iptables -A FORWARD -m conntrack --ctstate ESTABLISHED,RELATED
-j ACCEPT
```

iptables における接続追跡機能である conntrack の --ctstate オプションで

ESTABLISHED と RELATED のステートリストを指定して、それらに該当するパケットの通信を許可することによって [2], 既に確立済み (ESTABLISHED) または関連する (RELATED) 通信のパケットを許可し, 応答パケットを転送できるようにした. これによって server から windows への応答パケットの転送が可能となる.

```
# iptables -t nat -A PREROUTING -i eth1 -p tcp --dport 22 -j DNAT
--to-destination 192.168.100.2:22
```

nat テーブルの PREROUTING チェインに対して, eth1 から自身に対して送信された TCP 22 番ポートへのパケットを, 宛先 IP アドレス「192.168.100.2」, 宛先ポート「TCP 22 番」へと書き換える DNAT (宛先 IP アドレスの書き換え) の設定を行っている. これにより eth1 から自身の 22 番ポートへ送られたパケットは, サーバーの 22 番ポートに転送される.

```
# iptables -A FORWARD -i eth1 -o eth0 -p tcp -d 192.168.100.2
--dport 22 -j ACCEPT
```

filter テーブルの FORWARD チェインに対して, eth1 から送信された宛先 IP アドレスが「192.168.100.2」で宛先ポートが TCP 22 番のパケットの転送を許可する設定を行っている. これによって, DNAT で宛先 IP アドレスが変換されたパケットをサーバーの 22 番ポートに転送することが可能になる.

5. Windows から router へ通信を発生させ, server に一般権限ユーザでログインできることを確認する (必要に応じて known_hosts を編集して Host key 問題を解決する).

図 5 はデスクトップ PC から router への ssh 接続を試みた結果である.

ssh コマンドを実行すると, "remote host identification has changed!" と警告が発生していることがわかる. 一般に ssh のソフトウェアでは, ~/.ssh/known_hosts に接続先のサーバの公開鍵情報が保存される. 次回接続した際には, ここに保存された公開鍵と照合することによって同じサーバかどうかを検証する [4], このときに公開鍵の情報が一致しなければ, 今回得られたような警告文が出力される.

今回の場合においては以前に windows から router に対して ssh 接続した際に

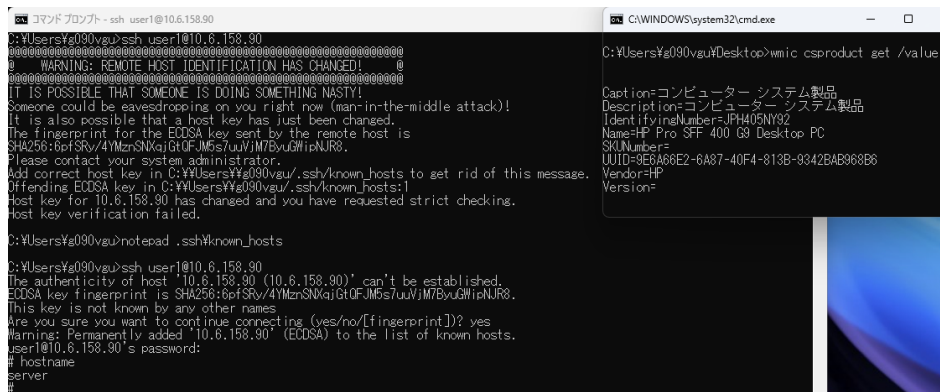


図5 デスクトップ PC から router への ssh 接続に成功する様子

router の公開鍵情報が `~/.ssh/known_hosts` に保存されており、今回も同じように router に対して ssh 接続のコマンドを実行したが、手順 4 において ssh のパケットを server に転送するよう設定したことで、server の公開鍵情報と以前に保存した router の公開鍵情報とで照合が行われ、一致せずに警告が発せられたと考えられる。ここでは記録されていた server の公開鍵情報を `~/.ssh/known_hosts` から削除することで対処した。結果として、最終的に windows から server に user1 としてログインできていることが確認できる。

6. router 上で Wireshark を起動し、Windows ↔ router および router ↔ server の両方のインターフェースをキャプチャ対象に設定する。
7. Windows から router への SSH ログイン過程のパケットを router の Wireshark で解析し、iptables による転送（IP アドレスやポート番号の書き換え）が観測できるパケットを特定する。

図 6 は router 上で Wireshark を起動し、Windows ↔ router および router ↔ server の両方のインターフェースをキャプチャ対象に設定した様子である。両方のインターフェースを流れるパケットを比較すると、プロトコルやパケット内の情報が酷似していることが確認できる。実際に、1 つパケットを抽出して比較した例を図 7 に示す。

図 7 に示すパケットは、それぞれ router の eth0, eth1 で観測された SSHv2 のパケットである。tcp のヘッダにおいて、フィールドの値 (Sequence Number(raw): 3645530960, Acknowledgment Number(raw): 1354975756

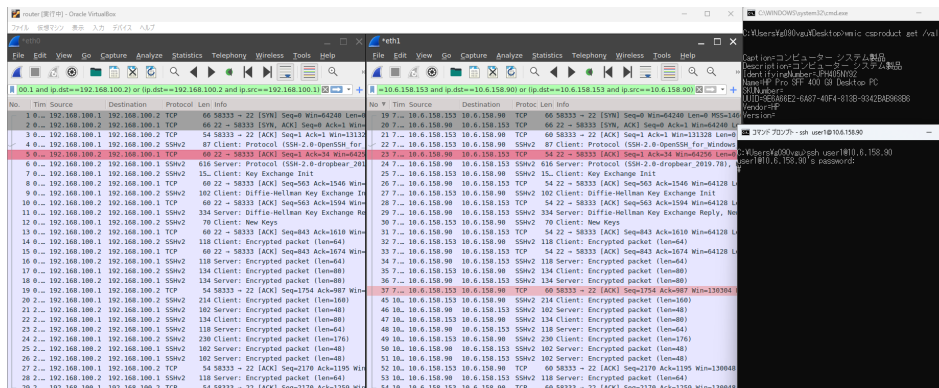


図 6 ssh ログイン時の Wireshark のキャプチャ結果

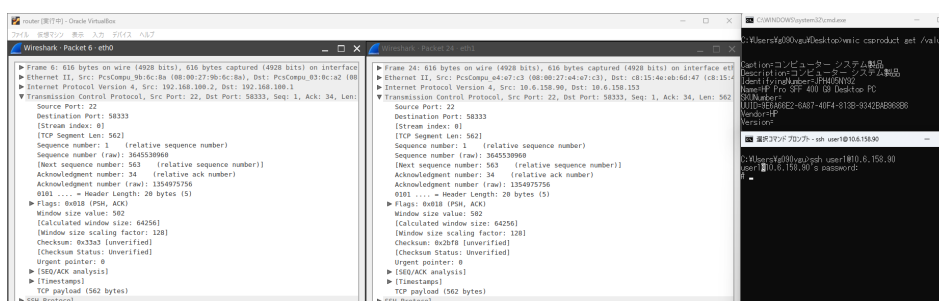


図 7 Windows ↔ router および router ↔ server を流れるパケットの比較例

など) が完全一致しており、転送されているものであるといえる。ここでは、server(192.168.100.2) の TCP 22 番ポートから router(192.168.100.1) の TCP 58333 番ポートへ送信されたパケットが、router(10.6.158.90) の TCP 22 番ポートから windows(10.6.158.153) の TCP 58333 番ポートへ送信されたパケットとして、宛先 IP アドレス、送信元 IP アドレスがともに書き換えられていることが分かる。

- router の組織内側インターフェースに対して、server から組織外側ネットワークへの通信が可能になるように iptables を設定する（例：POSTROUTING で SNAT/MASQUERADE を設定）。

図 8 は、手順 9 の要件を満たすための iptables の編集コマンドの実行結果である。ここで行った設定についてコマンドごとに説明を行う。

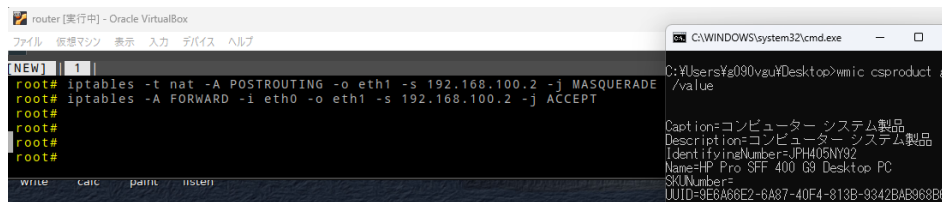


図 8 手順 9 の要件を満たすための iptables の編集コマンド

```
# iptables -t nat -A POSTROUTING -o eth1 -s 192.168.100.2 -j
MASQUERADE
```

nat テーブルの POSTROUTING チェインに対して、内部のサーバー (192.168.100.2) から eth1 を経由して外部へ出ていく通信の送信元 IP アドレスを、ルーターの外部 IP アドレスへと書き換える IP マスカレードの設定を行っている。

```
# iptables -A FORWARD -i eth0 -o eth1 -s 192.168.100.2 -j ACCEPT
```

filter テーブルの FORWARD チェインに対して、eth0 から受信して eth1 から送信される、サーバー (192.168.100.2) から外部へ向かうパケットの転送を許可する設定を行っている。

9. server から組織外ネットワークの端末へ通信を発生させ、router の Wireshark で解析し、iptables による変換（送信元 IP の書き換え等）を示すパケットを特定する。

図 9 は、server から windows に対して ping コマンドを実行した結果である。

windows からの応答が確認でき、server と windows との間で通信が可能であることが確認できる。

続いて、図 10 に server から windows に対して ping コマンドを実行した際にキャプチャされたパケットの一覧を示す。

左が eth0 でのキャプチャ結果、右が eth1 でのキャプチャ結果である。ここで、eth0 において server(192.168.100.2) から windows(10.6.158.153) に対して送信された ICMP のパケットが、eth1 においては、router の外部アドレス (10.6.158.90)

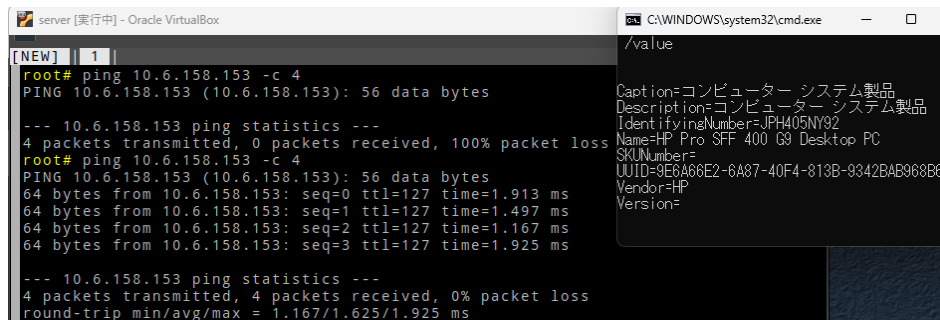


図 9 server から windows に対する ping コマンドの実行結果

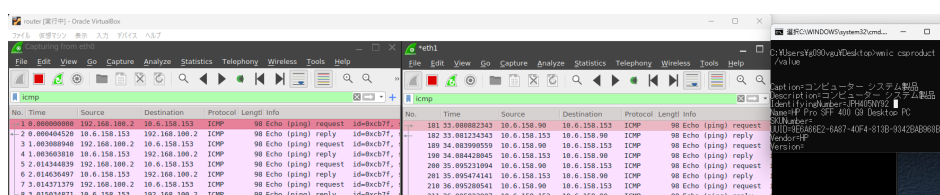


図 10 server から windows に対する ping コマンド実行時にキャプチャされたパケット一覧

から windows(10.6.158.153) に対してのパケットとして転送されていること、あるいはその逆のケースについても確認ができる。

10. 実験結果として、デスクトップ PC から server へログインできること、server から組織外ネットワークへ通信できることを示し、使用したすべての iptables 設定を列挙する。各設定がどのようにパケットを書き換え、転送処理を実現しているかを具体的に説明してレポートにまとめる。

手順 5, 9 においてそれぞれ転送によってデスクトップ PC から server にログインできることと組織外ネットワークに属する装置と通信ができることが確認できた。

また、手順 7, 9 においてパケットの iptables による転送処理 (パケットの書き換え) が実現されていることが確認できた。

ここでは各通信の実現に必要な設定についても各手順で述べたとおりであるが、server から組織外ネットワークへの通信に必要な設定について、デスクトップ PC から server にログインするために必要な設定と重複して、手順 9 で述べられていない部分があるため、ここではその点に対する補足を行う。

まずはカーネルパラメータの設定で、コマンドにすると以下のとおりである。

```
# sysctl -w net.ipv4.ip_forward=1
```

の部分で、これは2つのNIC間でのパケットの転送を許可するためのパラメータを設定するものであり、ルータとして機能させるためには必須の設定のためである。この実験ネットワークでは、具体的にrouterのeth1から届いたパケットをeth0から放出したい際、あるいはその逆の場合にホスト(router)内インターフェース間でのパケットの転送を実現するものである。

もう一点が、iptablesの設定で、コマンドにすると以下のものである。

```
# iptables -A FORWARD -m conntrack --ctstate ESTABLISHED,RELATED  
-j ACCEPT
```

これは、一度通信が確立された後の帰りのパケットを転送するための設定である。手順9では、組織内から組織外へパケットを送信する際に、routerの組織外とのインターフェースを用いて通信を行うための設定は行っているが、応答パケットを転送するための設定は行っていない。したがって、ここでこの設定を行うことによって、双方向の通信が実現される。

参考文献

- [1] kazmax. sysctl - システム管理コマンドの説明 - Linux コマンド集 一覧表. kazmax - Linux で自宅サーバー. Accessed: 2026-06-04. 2022. URL: <http://kazmax.zpp.jp>.
- [2] Linux man-pages project. iptables-extensions(8) — Linux manual page. iptables version 1.8.13. Accessed: 2026-06-04. man7.org. 2024. URL: <https://man7.org/linux/man-pages/man8/iptables-extensions.8.html>.
- [3] Rusty Russell and Netfilter Core Team. iptables(8) — IPv4/IPv6 のパケットフィルタと NAT の Version 1.4.21. Linux Manual Page, Section: iptables 1.4.21 (8). Netfilter Project. 2013. URL: <https://linuxjm.sourceforge.io/html/iptables/man8/iptables.8.html>.
- [4] S.O. known_hosts とは？. Zenn. Accessed: 2026-06-04. May 2025. URL: <https://zenn.dev>.